ELSEVIER

Future Generation Computer Systems 13 (1997) 99–115

FGCS
FUTURE GENERATION COMPUTER SYSTEMS

# Data mining and KDD: Promise and challenges

Usama Fayyad [a,*], Paul Stolorz [b,1]

[a] *Microsoft Research, One Microsoft Way, Redmond, WA 98052, USA*

[b] *Jet Propulsion Laboratory, California Institute of Technology, Pasadena, CA 91109, USA*

Received 5 March 1997; accepted 6 April 1997

**Abstract**

Databases are growing in size to a stage where traditional techniques for analysis and visualization of the data are breaking down. Data mining and knowledge discovery in databases (KDD) are concerned with extracting models and patterns of interest from large databases. Data mining techniques have their origins in methods from statistics, pattern recognition, databases, artificial intelligence, high performance and parallel computing, and visualization. In this article, we provide an overview of this growing multi-disciplinary research area, outline the basic techniques, and provide brief coverage of how they are used in some applications. We discuss the role of high performance and parallel computing in data mining problems, and we provide a brief overview of a few applications in science data analysis. We conclude by listing challenges and opportunites for future research.

*Keywords:* Data mining; Data analysis; Science data analysis; Overview article; Knowledge discovery in databases; Databases; Parallel data mining

## 1. From transactions to warehouses and beyond

With the widespread use of databases and the explosive growth in their sizes, individuals and organizations are faced with the problem of making use of this data. Traditionally, "use" of data has been limited to querying a reliable store via some well-circumscribed application or canned report-generating entity. While this mode of interaction is satisfactory for a wide-class of well-defined processes, it was not designed to support data exploration and ad hoc querying of the data. Now that capturing data and storing it has become easy and inexpensive, certain questions begin to naturally arise: Will these data help my business gain an advantage? How can we use historical data to build models of underlying processes that generated such data? Can we predict the behavior of such processes? How can we "understand" the data? These questions become particularly important in the presence of massive data sets since they represent a large body of information that is presumed to be valuable, yet is far from accessible. The currently available interfaces between humans and machines give little support to navigation, exploration, summarization, or modeling of large databases.

As transaction processing technologies were developed and became the mainstay of many business processes, great advances were achieved on the problems associated with reliable and accurate data capture. While transactional systems provided a solution to the problem of logging and book-keeping, there was little

* Corresponding author. E-mail: fayyad@microsoft.com.

[1] E-mail: pauls@aig.jpl.nasa.gov.


*PII* S0167-739X(97)00015-0

emphasis on supporting summarization, aggregation, and ad hoc querying over off-line stores of transactional data. A recent wave of activity in the database field, data warehousing, has been concerned with turning transactional data into a more traditional relational database that can be queried for summaries and aggregates of transactions. Data warehousing also includes the integration of multiple sources of data along with handling the host of problems associated with such an endeavor. These problems include: dealing with multiple data formats, multiple database management systems (DBMS), distributed databases, unifying data representation, data cleaning, and providing a unified logical view of an underlying collection of nonhomogeneous databases.

A data warehouse represents a large collection of data and in principle can provide views of the data that are not practical for individual transactional sources. For example, a supermarket chain may want to compare sales trends across regions at the level of products, broken down by weeks, and by class of store within a region. Such views are often precomputed and stored in special-purpose data stores that provide a multi-dimensional front-end to the underlying relational database and are sometimes called multi-dimensional databases.

Data warehousing is the first step in transforming a database system from a system whose primary purpose is reliable storage to one whose primary use is decision support (Fig. 1). A closely related area is called on-line analytical processing (OLAP) named after principles first advocated by Codd [4].

The current emphasis of OLAP systems is on supporting query-driven exploration of the data warehouse. Part of this entails precomputing aggregates along data "dimensions" in the multi-dimensional data store. Because the number of possible aggregates is exponential in the number of "dimensions", much of the work in OLAP systems is concerned with deciding which aggregates to precompute and how to derive


Fig. 1. The evolution of the view of a database system.

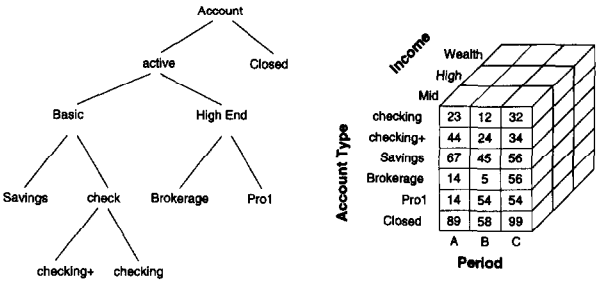



Fig. 2. An example of multi-dimensional view of relational data.

other aggregates (or estimate them reliably) from the precomputed projections. Fig. 2 illustrates one such example for data representing summaries of financial transactions in branches of a nationwide bank. The attributes (dimensions) have an associated hierarchy which defines how quantities are to be aggregated (rolled-up) as one moves to higher levels in the hierarchy.

Note that the multi-dimensional store may not necessarily be materialized as in principle it could be derived dynamically from the underlying relational database. For efficiency purposes, some OLAP systems employ "lazy" strategies in precomputing summaries and incrementally build up a cache of aggregates.

While OLAP systems enable the execution of queries that would normally be very expensive to execute, and while they provide a more convenient environment for manipulating and visualizing data, they are dependent on human-driven exploration and rely on the analyst to formulate and test new hypothesis in the data. Exploration is guided by queries issued by the user that specify level in dimension hierarchy (e.g., "drill down" operations to move into a lower level of detail). Inference or modeling is relegated completely to the analyst who is expected to "find" patterns of interest via visualization in low-dimensional projections (or summaries) of data.

## 2. Why data mining?

Unlike OLAP, data mining techniques allow for the possibility of computer-driven exploration of the data.

This opens up the possibility for a new way of interacting with databases: specifying queries at a much more abstract level than SQL permits. It also facilitates data exploration for problems that, due to high-dimensionality, would otherwise be very difficult to explore by humans, regardless of difficulty of use of, or efficiency issues associated with, SQL.

A problem that has not received much attention in database research is the *query formulation problem*: how can we provide access to data when the user does not know how to describe the goal in terms of a specific query? Examples of this situation are fairly common in decision support situations. One example is a credit card or telecommunications company that would like to query its database of usage data for records representing fraudulent cases. Another example would be a scientist dealing with a large body of data who would like to request a catalog of events of interest appearing in the data. Such patterns, while recognizable by human analysts on a case by case basis, are typically very difficult to describe in a SQL query or even as a computer program in a stored procedure. A more natural means of interacting with the database is to state the query by example. In this case, the analyst would label a training set of examples of cases of one class versus another and let the data mining system build a model for distinguishing one class from another. The system can then apply the extracted classifier to search the full database for events of interest. This is typically much more feasible than detailed modeling of the causal mechanisms underlying the phenomena, because examples are usually easily available, and humans find it natural to interact at the level of cases.

Another major problem which data mining could help alleviate is the fact that humans find it particularly difficult to visualize and understand a large data set. Data can grow along two dimensions: the number of fields (also called dimensions or attributes) and the number of cases. Human analysis and visualization abilities do not scale to high dimensions and massive volumes of data. A standard approach to dealing with high-dimensional data is to project it down to a very low-dimensional space and attempt to build models in this simplified subspace. As the number of dimensions grow, the number of choice combinations for dimensionality reduction explode. Furthermore, a projection to lower dimensions could easily transform an otherwise solvable discrimination problem into one that is impossible to solve. If the analyst is trying to explore models, then it becomes infeasible to go through the various ways of projecting the dimensions or selecting the right subsamples (reduction along columns and rows). An effective means to visualize data would be to employ data mining algorithms to perform the appropriate reductions. For example, a clustering algorithm could pick out a distinguished subset of the data embedded in a high-dimensional space and proceed to select a few dimensions to distinguish it from the rest of the data or from other clusters. Hence, a much more effective visualization mode could be established: one that may enable an analyst to find patterns or models which may otherwise remain hidden in the high-dimensional space.

Another factor that is turning data mining into a necessity is that the rates of growth of data sets exceeds by far any rates that traditional "manual" analysis techniques can cope with. If one is to utilize the data in a timely manner, it will be necessary to go beyond traditional data analysis approaches, for which most of the data must remain unused. Such a scenario is not realistic in any competitive environment where others who do utilize data resources better will gain a distinct advantage. This sort of pressure is present in a wide variety of organizations, spanning the spectrum from business, to science, to government. It is leading to serious reconsideration of data collection and analysis strategies that are nowadays causing huge "write-only" data stores to accumulate.

## 3. KDD and data mining

The term *data mining* is often used as a synonym for the process of extracting useful information from databases. In this paper, as in [7, Chap. 1], we draw a distinction between the latter, which we call KDD, and "data mining". The term *data mining* has been mostly used by statisticians, data analysts, and the management information systems (MIS) communities. It has

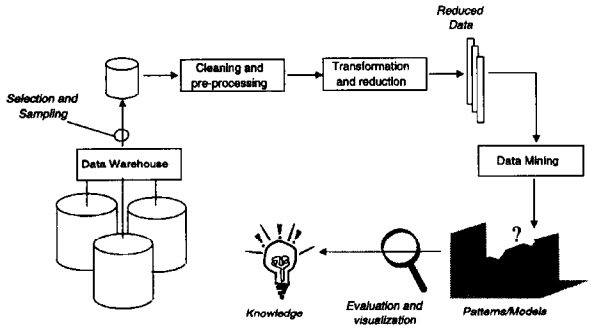



Fig. 3. An overview of the steps comprising the KDD process.

also gained popularity in the database field. The earliest uses of the term come from statistics and the usage in most cases was associated with negative connotations of blind exploration of data without a priori hypotheses to verify. However, notable exceptions can be found. For example, as early as 1978, Leamer [16] used the term in a positive sense in a demonstration of how generalized linear regression can be used to solve problems that were very difficult for humans and for traditional statistical techniques of that time to solve. The term KDD was coined at the first KDD workshop in 1989 [19] to emphasize that "knowledge" is the end product of a data-driven discovery.

In our view KDD refers to the overall *process* of discovering useful knowledge from data while *data mining* refers to a particular *step* in this process. Data mining is the application of specific algorithms for extracting patterns from data. The additional steps in the KDD process, such as data preparation, data selection, data cleaning, incorporating appropriate prior knowledge, and proper interpretation of the results of mining, are essential to ensure that useful knowledge is derived from the data. Blind application of data mining methods (rightly criticized as "data dredging" in the statistical literature) can be a dangerous activity easily leading to discovery of meaningless patterns. We give an overview of the KDD process in Fig. 3. Note that in the KDD process, one typically iterates many times over previous steps and the process is fairly messy with plenty of experimentation. For example, one may select, sample, clean, and reduce data only to discover after mining that one or several of the previous steps need to be redone. We have omitted arrows illustrating these potential iterations to keep the figure simple.

### 3.1. Basic definitions

We adopt the definitions of KDD and data mining provided in [7, Chap. 1].

*Knowledge discovery in databases* is the *nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data.*

Here *data* is a set of facts (e.g., cases in a database) and *pattern* is an expression in some language representing a parsimonious description of a subset of the data or a model applicable to that subset. Hence, in our usage here, extracting a *pattern* also designates fitting a model to data, finding structure from data, or in general any high-level description of a set of data. The term *process* implies that KDD is comprised of many steps, which involve data preparation, search for patterns, knowledge evaluation, and refinement, all repeated in multiple iterations. By *nontrivial* we mean that some search or inference is involved, i.e., it is not a straightforward computation of predefined quantities like computing the average value of a set of numbers. The discovered patterns should be *valid* on new data with some degree of certainty. We also want patterns to be *novel* (at least to the system, and preferably to the user) and *potentially useful*, i.e., lead to some benefit to the user/task. Finally, the patterns should be *understandable*, if not immediately then after some post-processing.

While it is possible to define quantitative measures for *certainty* (e.g., estimated prediction accuracy on new data) or *utility* (e.g., gain perhaps in dollars saved due to better predictions or speed-up in response time of a system), notions such as *novelty* and *understandability* are much more subjective. In certain contexts understandability can be estimated by simplicity (e.g., the number of bits to describe a pattern). An important notion, called *interestingness* (e.g., see [20] and references within), is usually taken as an overall measure of pattern value, combining validity, novelty, usefulness, and simplicity. Interestingness functions can be explicitly defined or can be manifested implicitly via

an ordering placed by the KDD system on the discovered patterns or models.

Hence, what we mean by the term "knowledge" in the KDD context can be characterized in a very simplified sense. A pattern can be considered as *knowledge* if it exceeds some interestingness threshold. This is by no means an attempt to define "knowledge" in the philosophical or even the popular view. It is purely user-oriented, domain-specific, and determined by whatever functions and thresholds the user chooses.

*Data mining* is a step in the KDD process consisting of applying computational techniques that, under acceptable computational efficiency limitations, produce a particular enumeration of patterns (or models) over the data [7, Chap. 1]

Note that the space of patterns is often infinite, and the enumeration of patterns involves some form of search in this space. Practical computational constraints place severe limits on the subspace that can be explored by a data mining algorithm.

The data mining component of the KDD process is concerned with the algorithmic means by which patterns are extracted and enumerated from data. The overall KDD process (Fig. 3) includes the *evaluation* and possible *interpretation* of the "mined" patterns to determine which patterns may be considered new "knowledge".

### 3.2. Related fields

Many research communities are strongly related to KDD. For example, by our definition, all work in classification and clustering in statistics, pattern recognition, neural networks, machine learning, and databases would fit under the data mining step. In addition to exploratory data analysis (EDA) [23], statistics overlaps with KDD in many other steps including data selection and sampling, preprocessing, transformation, and evaluation of extracted knowledge. For more details on the strong links between statistics and KDD, see [7, Chap. 4; 8].

The database field is of fundamental importance to KDD. The efficient and reliable storage and retrieval of the data, as well as issues of flexible querying and query optimization, are important enabling techniques. In addition, contributions from the database research literature in the area of data mining are beginning to appear. OLAP is an evolving field with very strong ties to databases, data warehousing, and KDD. While the emphasis in OLAP is still primarily on data visualization and query-driven exploration, automated techniques for data mining can play a major role in making OLAP more useful and easier to apply.

Other related fields include optimization (in search), high-performance and parallel computing (as we illustrate later in this paper), knowledge modeling, the management of uncertainty, and data visualization. Data visualization can contribute to effective EDA and visualization of extracted knowledge. Data mining can enable the visualization of patterns hidden deep within the data and embedded in much higher-dimensional spaces, as discussed in Section 2.

While KDD is fundamentally a statistical endeavor, statisticians have not as a rule focussed on considering issues related to large databases. In addition, historically, the majority of work in statistics has been primarily focussed on hypothesis verification as the primary mode of data analysis (which is certainly no longer true now). The de-coupling of database issues (storage and retrieval) from analysis issues is also a culprit. Furthermore, compared with techniques that data mining draws on from pattern recognition, machine learning, and neural networks, the traditional approaches in statistics perform little search over models and parameters (again with notable recent exceptions). KDD is concerned with formalizing and encoding aspects of the "art" of statistical analysis and making analysis methods easier to use by those who own the data, regardless of whether they have the pre-requisite knowledge of the techniques being used. A marketing person interested in segmenting a database may not have the necessary advanced degree in statistics to understand and use the literature or the library of available routines. We do not dismiss the dangers of blind mining, and we warn that it can easily deteriorate to data dredging. However, the strong need for analysis aids in this data-overloaded society needs to be addressed.

In response to the growing need, the KDD community emerged. The first KDD workshop [19] was

held in Detroit in August 1989. Workshops followed in 1991, 1993, with the last workshop in 1994 which then evolved into annual international conferences starting with KDD-95 in 1995.[2]

## 4. Data mining methods

Data mining techniques can be divided into five classes of methods. These methods are listed below. While much of these techniques have been historically defined to work over memory-resident data (typically read from flat files), some of these techniques are beginning to be scaled to operate on databases. Examples in classification (Section 4.1) include decision trees [17] and in summarization (Section 4.3) association rules [7, Chap. 12].

### 4.1. Predictive modeling

The goal is to predict some field(s) in a database based on other fields. If the field being predicted is a numeric (continuous) variable (such as a pysical measurement of, e.g., *height*) then the prediction problem is a *regression* problem. If the field is categorical then it is a *classification* problem. There is a wide variety of techniques for classification and regression. The problem in general is cast as determining the most likely value of the variable being predicted given the other fields (inputs), the training data (in which the target variable is given for each observation), and a set of assumptions representing one's prior knowledge of the problem.

Linear regression combined with nonlinear transformation on inputs could be used to solve a wide range of problems. Transformation of the input space is typically the difficult problem requiring knowledge of the problem and quite a bit of "art". In classification problems this type of transformation is often referred to as "feature extraction".

In classification the basic goal is to predict the most likely state of a categorical variable (the class). This is fundamentally a density estimation problem. If one can estimate the probability that the class $C = c$, given the other fields $X = x$ for some feature vector $x$, then one could derive this probabilty from the joint density on $C$ and $X$. However, this joint density is rarely known and very difficult to estimate. Hence, one has to resort to various techniques for estimating. These techniques include:

1. Density estimation, e.g., kernel density estimators [5] or graphical representations of the joint density [11].
2. Metric-space based methods: Define a distance measure on data points and guess the class value based on proximity to data points in the training set. For example, the $K$-nearest-neighbor method [5].
3. Projection into decision regions: Divide the attribute space into decision regions and associate a prediction with each. For example, linear discriminant analysis finds linear separators, while decision tree or rule-based classifiers make a piecewise constant approximation of the decision surface. Neural nets find nonlinear decision surfaces.

### 4.2. Clustering

Also known as segmentation, clustering does not specify fields to be predicted but targets separating the data items into subsets that are similar to each other. Since unlike classification we do not know the number of desired "clusters", clustering algorithms typically employ a two-stage search: An outer loop over possible cluster numbers and an inner loop to fit the best possible clustering for a given number of clusters. Given the number $K$ of clusters, clustering methods can be divided into three classes:

1. *Metric-distance based methods*: A distance measure is defined and the objective becomes finding the best $K$-way partition such as cases in each block of the partition are closer to each other (or centroid) than to cases in other clusters.
2. *Model-based methods*: A model is hypothesized for each of the clusters and the idea is to find the best fit of that model to each cluster. If $M_k$ is the model hypothesized for cluster $k$, then one way to score the fit of a model to a cluster is via the likelihood:

[2] For more information on the KDD conference, see http://www-aig.jpl.nasa.gov/kdd97.

$$\text{Prob}(M_k|D) = \text{Prob}(D|M_k)\frac{\text{Prob}(M_k)}{\text{Prob}(D)}.$$

The prior probability of the data $D$, Prob($D$) is a constant and hence can be ignored for comparison purposes, while Prob($M_k$) is the prior assigned to a model. In maximum likelihood techniques, all models are assumed equally likely and hence this term is ignored. A problem with ignoring this term is that more complex models are always preferred and this leads to overfitting the data.

3. *Partition-based methods*: Basically enumerate various partitions and then score them by some criterion. The above two techniques can be viewed as special cases of this class. Many techniques in the AI literature fall under this category and utilize ad hoc scoring functions.

### 4.3. Data summarization

Sometimes the goal is to simply extract compact patterns that describe subsets of the data. There are two classes of methods which represent taking horizontal (cases) or vertical (fields) slices of the data. In the former, one would like to produce summaries of subsets: e.g., producing sufficient statistics, or logical conditions that hold for subsets. In the latter case, one would like to predict relations between fields. This class of methods is distinguished from the above in that rather than predicting a specified field (e.g., classification) or grouping cases together (e.g., clustering) the goal is to find relations between fields. One common method is called association rules [7, Chap. 12]. Associations are rules that state that certain combinations of values occur with other combinations of values with a certain frequency and certainty. A common application of this is market basket analysis where one would like to summarize which products are bought with what other products.

### 4.4. Dependency modeling

Insight into data is often gained by deriving some causal structure within the data. Models of causality can be probabilistic (as in deriving some statement about the probability distribution governing the data) or they can be deterministic as in deriving functional dependencies between fields in the data [19]. Density estimation methods, in general, fall under this category, so do methods for explicit causal modeling (e.g., [9,11].

### 4.5. Change and deviation detection

These methods account for sequence information, be it time-series or some other ordering (e.g., protein sequencing in genome mapping). The distinguishing feature of this class of methods is that ordering of observations is important and must be accounted for.

## 5. Scalability and high-performance computing aspects of KDD

Because large data sets and databases are ubiquitous in data mining applications, a great deal of attention must clearly be devoted to the issue of constructing and fielding algorithms that scale appropriately as data volumes grow. In general, the growth in algorithm complexity must be limited to a low-order polynomial of the problem size. In fact, for massive data sets, an algorithm whose complexity is as low as $O(n^2)$ is considered infeasible. The goal should be achieving linear or even sublinear complexity (using clever sampling techniques whenever possible).

However, careful control of algorithm complexity on serial machines is not enough, by itself, to bring all the databases of importance within the range of straightforward serial computers. Scalable high-performance computing environments are essential for a large class of important problems for a number of different reasons. They include the following:

*Super-linear algorithm scaling.* We would in principle like to only use methods that scale linearly with problem size (or less). However, there are a number of data mining methods that to date outperform all their competitors in terms of accuracy within important problem domains, but which scale worse than

linearly. Two prominent examples occur in computational biology and astrophysics.

In the latter field, the dominant method for reliably computing the evolution of $N$ galaxies scales as $N \log N$. This is already a lot better than the exact method based upon Newtonian mechanics, which scales atrociously as $N^2$. But it remains problematic when tens of millions of particles must be tracked in the simulation. In this case there is no alternative but to resort to parallel machines to handle the CPU and memory requirements of the largest scale problems [18].

In computational biology, a standard method, based upon dynamic programming, for predicting the secondary structure of RNA molecules scales as $N^3$ with respect to CPU power, and as $N^2$ with respect to memory. In view of the method's reliability, it is important to obtain results using this method on long RNA sequences such as HIV (10 000 nucleotides). This can only be done using a parallel supercomputer, with the memory requirements becoming the main driver in the analysis [12].

*Heterogenous distributed data repositories.* There is an enormous amount of data of all kinds collected over the years and stored in different formats across a number of distributed locations. This is the case even for data sets that are relatively homogeneous. The increasingly urgent need to fuse data obtained from completely different sensors and input streams compounds the problems further. An inherently distributed computing approach is required to deal with this situation efficiently.

*Low-latency applications.* As data mining methods mature, we can expect that CPU demands will eventually be matched to data size. However, large data volumes will then dominate the problem by presenting enormous I/O demands, particularly in cases where low-latency is important. Scalable I/O resources and bandwidth to memory, as opposed to simply volume, then become at least as important as CPU power and total RAM availability.

*Exhaustive analysis versus sampling.* Statistical clustering methods such as $K$-means and EM algorithms applied to Gaussian mixtures are very popular methods that work extremely well with limited data set sizes. It is conceivable that they can be scaled up to massive data sets by careful sampling (they scale too poorly to be applied naively to large data sets on workstations). But this approach runs a high risk that important clusters and outliers will be either completely missed or otherwise mishandled in the resulting analysis. A much safer approach is to decompose the methods on scalable high-performance platforms, which increases by two orders of magnitude the accessible data set sizes before it becomes necessary to resort to sampling.

## 6. Applications in science data analysis

KDD techniques have had some of their most prominent early successes in the scientific realm. These early examples illustrate the promise of KDD, while displaying the need to incorporate specific domain knowledge as well. We shall briefly review four scientific case studies in order to illustrate the contribution and potential of KDD for science data analysis. For each case, the focus will primarily be on impact of application, reasons why KDD systems succeeded, and limitations and future challenges of the approach.

### 6.1. Sky survey cataloging

The second Palomar Observatory Sky Survey is a major undertaking that over six years to complete. The survey consists of 3 terabytes of image data containing an estimated two billion sky objects. The 3 000 photographic images are scanned into 16-bit/pixel resolution digital images at 23 040 × 23 040 pixels per image. The basic problem is to generate a survey catalog which records the attributes of each object along with its class: star or galaxy. The attributes are defined by the astronomers. Once basic image segmentation is performed, 40 attributes per object are measured. The measurement process is fairly straightforward to automate. The problem is identifying the class of each object. Once the class is known, astronomers can conduct all sorts of scientific analyses such as probing Galactic structure from star/galaxy counts, modeling

evolution of galaxies, and studying the formation of large structure in the universe [25]. To achieve these goals the SKICAT system (Sky Image Cataloging and Analysis Tool) [26] was developed.

The problem of obtaining class labels is significant. The majority of objects in each image are faint objects whose class cannot be determined by visual inspection or classical computational approaches in astronomy. The goal was to classify objects that are at least one isophotal magnitude fainter than objects classified in previous comparable surveys. The problem was tackled using decision tree learning algorithms [7, Chap. 19] to accurately predict the classes of objects. Accuracy of the procedure was verified by using a very limited set of high-resolution CCD images as ground truth.

By extracting rules via statistical optimization over multiple trees [7, Chap. 19] it was possible to achieve 94% accuracy on predicting sky object classes. Since the approach could reliably classify objects that are one magnitude fainter than previous techniques, this resulted in tripling the size of data that is classified (usable for analysis). With the 300% increase in available data, astronomers were able to extract much more out of the data in terms of new scientific results [26]. In fact, recently this helped a team of astronomers discover 16 new high red-shift quasars in the universe in at least one order of magnitude less observation time [14]. These objects are extremely difficult to find, and are some of the farthest (hence oldest) objects in the universe. They provide valuable and rare clues about the early history of our universe.

SKICAT was successful for the following reasons:

1. The astronomers solved the feature extraction problem: the proper transformation from pixel space to feature space. This transformation implicitly encodes a significant amount of prior knowledge to the system.
2. Within the 40-dimensional feature space, at least eight dimensions are needed for accurate classification. Hence it was difficult for humans to discover which eight of the 40 to use, let alone how to use them in classification. Data mining methods contributed in this case by solving the difficult classification problem.
3. Manual approaches to classification were simply not feasible. Astronomers needed an automated classifier to make the most out of the data.
4. Decision tree methods, although involving blind greedy search, proved to be an effective tool for finding the important dimensions for this problem.

Directions being pursued now involve the unsupervised learning (clustering) version of the problem. Unusual or unexpected clusters in the data might be indicative of new phenomena, perhaps even a new discovery. In a database of hundreds of millions of objects, automated analysis techniques are a necessity since browsing the feature vectors manually would only be possible for a small fraction of the survey. The idea is to pick out subsets of the data that look interesting, and ask the astronomers to focus their attention on those, perhaps perform further observations, and explain why these objects are different. A difficulty here is that new classes are likely to be a rare occurrence in the data, so algorithms need to be tuned to looking for small interesting clusters rather than ignoring them.

### 6.2. *Finding volcanoes on Venus*

The Magellan spacecraft orbited the planet Venus for over five years and used synthetic aperture radar (SAR) to map the surface of the planet penetrating the gas and cloud cover that permanently obscures the surface in the optical range. The resulting data set is a unique high-resolution global map of an entire planet. In fact, we have more of the planet Venus mapped at the 75 m/pixel resolution than we do of our own planet Earth's surface (since most of Earth's surface is covered by water). This data set is uniquely valuable because of its completeness and because Venus is similar to Earth in size. Learning about the geological evolution of Venus could offer valuable lessons about Earth and its history.

The sheer size of the data set prevents planetary geologists from effectively exploiting its content. For example, the first pass of Venus (completed in the first two years) resulted in over 30 000 images, each having $10^6$ pixels ($1000 \times 1000$). The data set was released on 100 CD-ROMs and is available to anyone who is interested. Lacking the proper tools to analyze

this data, geologists did something very predictable: they simply examined browse images, looked for large features or gross structure, and cataloged/mapped the large-scale features of the planet. In effect, scientists operated at a much lower resolution than the data itself, ignoring the potentially valuable high-resolution data actually collected. Given that it took billions of dollars to design, launch, and operate the sensing instruments, it was a priority for NASA to insure that the data be exploited as fully as possible.

To help a group of geologists at Brown University analyze this data set [1], the JPL adaptive recognition tool (JARtool) was developed [3]. The idea behind this system is to automate the search for an important feature on the planet, small volcanoes, by training the system via examples. The geologists would label volcanoes on a few (say 30–40) images, and the system would automatically construct a classifier that would then proceed to scan the rest of the image database and attempt to locate and measure the estimated one million small volcanoes.

Locating and cataloging the small volcanoes would enable the geologists to infer a lot about the geologic evolution of the planet and the processes it used to resurface itself. This is an excellent example of the wide gap between the raw collected data (pixels) and the level at which scientists operate (catalogs of objects). In this case, unlike in SKICAT, the mapping from pixels to features would have to be done by the system. Hence, little prior knowledge is provided to the data mining system.

Using an approach based on matched filtering for focus of attention (triggering on any candidates with a high false alarm rate), followed by feature extraction based on projecting the data onto the dominant eigenvecotrs in the training data, using singular value decomposition, and then classification learning to distinguish true detections from false alarms, JARtool can match scientist performance for certain classes of volcanoes (high probability volcanoes versus ones which scientists are not sure about) [3].

The use of data mining methods here was well-motivated because:

1. Scientists did not know much about image processing or about the SAR properties. Hence, they could easily label images but programming classifiers was not really feasible. Hence, a training-by-example framework is natural and justified.
2. Fortunately, as is often the case with cataloging tasks, there was little variation in illumination and orientation of objects of interest. Hence, the mapping from pixels to features can be performed automatically.
3. The geologists did not have any other easy means for finding the small volcanoes, hence, they were motivated to cooperate by providing training data and other help.
4. The result is the extraction of valuable information from an expensive data set. Also, the adaptive approach (training by example) is flexible and would in principle allow us to reuse the basic approach on other problems.

With the proliferation of image databases and digital libraries, data mining systems that are capable of searching for content are becoming a necessity. In dealing with images, the train-by-example approach, i.e., querying for "things that look like this" is a natural interface since humans can visually recognize items of interest, but translating those visual intuitions into pixel-level algorithmic contraints is difficult to do. Future work on JARtool is proceeding to extend it to other applications such as classification and cataloging of sun spots.

### *6.3. Earth geophysics – Earthquake photography from space*

A major problem facing scientists in domains such as remote sensing is the fact that important signals about temporal processes are often buried within noisy image streams, requiring the application of systematic statistical inference concepts in order for raw image data to be transformed into scientific understanding.

One class of problems that require inference in this way is the measurement of subtle changes in images. Consider for example the case of two images taken before and after an earthquake, at a pixel resolution of say 10 m. If the earthquake fault motions vary only up to 5 or 6 m in magnitude, a relatively common scenario, then it is essentially impossible to describe and

measure the fault motion by simply comparing the two images manually (or even by naive differencing by computer). However, by repeatedly registering different local regions of the two images, a task that is known to be do-able to subpixel precision, it is possible to infer the direction and magnitude of ground motion due to the earthquake. The fundamental concept is broadly applicable to many data mining situations in the geosciences and other fields, including earthquake detection, continuous monitoring of crustal dynamics and natural hazards, target identification in noisy images and so on.

This type of data mining algorithm needs to simultaneously address three distinct problems in order to be successful, namely (1) the design of a statistical inference engine that can reliably infer the fundamental processes to acceptable precision, (2) development and implementation of scalable algorithms on scalable platforms so that it can be used on increasingly massive data sets, and (3) construction of an automatic and reasonably seamless system that can be used by domain scientists on a large number of data sets.

One example of such a geoscientific data mining system is Quakefinder [21], which automatically detects and measures tectonic activity in the earth's crust by examination of satellite data. Quakefinder has been used to automatically map the direction and magnitude of ground displacements due to the 1992 Landers earthquake in Southern California, over a spatial region of several hundred square kilometers, at a resolution of 10 m, to a (subpixel) precision of 1 m. It is implemented on a 256-node Cray T3D parallel supercomputer to ensure rapid turn-around of scientific results. The issues of scalable algorithm development and their implementation on scalable platforms that were addressed here are in fact quite general, and are likely to impact the great majority of future data mining efforts geared to the analysis of genuinely massive data sets.

The system addressed a definite scientific need, as there was previously no area-mapped information about two-dimensional tectonic processes available at this level of detail. In addition to automatically measuring known faults, the system also enabled a form of automatic knowledge discovery by indicating novel unexplained tectonic activity away from the primary Landers faults that has never before been observed.

Quakefinder was successful for the following reasons:

1. It was based upon an integrated combination of techniques drawn from statistical inference, massively parallel computing and global optimization.
2. Scientists were able to provide a concise description of the fundamental signal recovery problem.
3. Portions of the task based upon statistical inference were straightforward to automate and parallelize, while still ensuring accuracy.
4. The relatively small portions of the task not so easily automated, such as careful measurement of fault location based on a computer-generated displacement map, are accomplished very quickly and accurately by humans in an interactive environment.

The overall system provides a fast, reliable, high-precision change analyzer able to measure earthquake fault activity to high-resolution. It enables geologically important events to be extracted and cataloged automatically from large data streams, mining large-scale pixel imagery for scientifically meaningful signals while discarding uninteresting information. It is currently being extended to study sand-dune motions on the earth and on the surface of Mars, as well as to look for tectonic motion on the ice-covered surface of Jupiter's moon Europa. We believe that its success in these domains also points the way to a number of broader data mining calculations that can directly and automatically measure important temporal processes to high precision from massive data sets, including "the measurement of contine" ous processes.

The limitations of the approach include the fact that it relies upon successive images being "similar enough" to each other to allow inference based upon cross-correlation measures. This is not always the case in regions where, for example, vegetation growth is vigorous. The method also requires relative cohesive ground motions over a number of pixels (although these motions can in principle be of any magnitude, provided that the relevant CPU resources are available). Note that the method does not require images to be preregistered to a common grid in order to work-on the contrary, efforts to register successive images

will succeed only in corrupting the fundamental signal of interest. Points such as this are often overlooked in the headlong rush to "fuse" and otherwise render image information usable. The success of Quakefinder is a powerful reminder that raw data is by far the most trustworthy information on which to base any KDD system, and that cleaned and scrubbed "data" are already models of the system of interest, albeit low-level ones, and should never be confused with the raw data itself.

### 6.4. Atmospheric science – Conquest

Analysis of atmospheric data is a classic area where processing and data collection power has far outstripped our ability to interpret the results. The mismatch between pixel-level data and scientific language that understands spatio-temporal patterns such as cyclones and tornadoes is huge. A collaboration between scientists at JPL and UCLA has developed CONQUEST (COncurrent QUErying Space and Time) [22], a scientific information system implemented on parallel supercomputers, to bridge this gap.

Parallel testbeds (MPPs) were employed by Conquest to enable rapid extraction of spatio-temporal features for content-based access to large scientific data sets. In some cases, the features are known beforehand. One well-known example is the detection of cyclone tracks, which are extremely distinctive climatological features. Other times indexable features are hidden in the enormous mass of data. Hence, one of the goals here has been the development of "learning" algorithms on MPPs which look for novel patterns, event clusters or correlations on a number of different spatial and temporal scales. MPPs are also used by CONQUEST to service user queries requiring complex and costly computations on large data sets.

A prototype version of CONQUEST has been used to study atmospheric model data generated by the UCLA Atmospheric Sciences Department on a ($4^\circ \times 5^\circ$ resolution) grid. This model generates Gigabytes of data covering several years of simulated time. We have implemented parallel queries concerning the presence, duration and strength of extratropical cyclones and distinctive "blocking features" in the atmosphere, which can scan through this data set in minutes. Other features of great interest are being added, including the detection and analysis of ocean currents and eddies. Upon extraction, the features are stored in a relational database (Postgres). This content-based indexing dramatically reduces the time required to search the raw data sets of atmospheric variables when further queries are formulated.

The system also features parallel implentations of singular value decomposition and neural network pattern recognition algorithms, in order to identify spatio-temporal features as a whole, in contrast to the separate treatment of spatial and temporal information components that has often been used in the past to study atmospheric data. This approach requires parallel supercomputers as a fundamental component, since the computational demands exceed those that can be supplied by workstations. The long-term goal is to develop a flexible, extensible, and seamless environment for scientific data analysis, which can be applied ultimately to a number of entirely different scientific domains. This would have great utility in a number of different fields outside the area of geophysical databases alone.

## 7. The role of high-performance computing in KDD

To illustrate the role of high-performance computing (HPC) in two slightly different contexts, we show the motivation for the parallel approach taken by the Quakefinder system and the RNA folding problem mentioned above.

### 7.1. Scalable decomposition of the Quakefinder application

Let us analyze briefly the role of high performance computers in the Quakefinder application described above. The core computation in the approach is that of ground motion inference for a given image pixel. On machines of the workstation class, several hundred such ground motion vectors can be calculated in

a matter of hours for disjoint local areas of ground centered by the pixel of interest. However, when area-based maps of ground motion are needed, requiring a separate caculation for each and every pixel, workstations are no longer sufficient. For example, for even relatively small images of 2000 × 2000 pixels, four million vectors must now be computed, raising the computational demands by four orders of magnitude over what would be needed for disjoint 100 × 100 area templates. The resulting calculation is not accessible to workstations in any reasonable time frame. Quakefinder was therefore implemented on a 256-node Cray T3D at JPL. The T3D is a massively parallel distributed memory architecture consisting of 256 computing nodes, each based on a DEC Alpha processor running at 150 MHz. The nodes are arranged as three-dimensional tori, allowing each node to communicate directly with up to six neighboring nodes of the machine.

MIMD parallel architectures such as the T3D turn out to be ideal architectures on which to implement many image analysis algorithms, including Quakefinder. The best decomposition consists of simply assigning different spatial portions of each image to different nodes of the machine. The vast majority of the calculation can then proceed independently on each node, with very limited communication. This results in a highly efficient parallel algorithm.

A small amount of communication does of course need to be performed periodically. The storage required for some templates will occasionally overlap boundaries between nodes. A standard technique in parallel decomposition handles this problem by tolerating a small amount of memory redundancy. Each node is simply assigned a slightly larger area of the image at the start of the calculation to allow for these overlaps. The overhead required is very small. Thereafter the calculation can proceed entirely in parallel.

Note that the core challenge here is to think of the T3D as an interactive data mining tool, rather than a long-term batch processing tool. The fundamental task does not follow the standard parallel computing model of a large initial coding investment, followed by a long T3D production run for weeks or months. Instead, it consists of rapid prototyping of an algorithm directly for the T3D, application of this code to the data set, and the recovery of an answer to a given query in a matter of two to three hours. The issue of the relevant time scale for interactivity is paramount here. Because an answer to a query was provided by the T3D in two hours, it was possible to debug the algorithm, understand what the first-cut algorithm was doing on the data, and run several different processes in a single day to understand the behavior of the method on real data. This meant that in a matter of a few weeks it was possible to produce a final displacement map by iteratively getting rid of mistakes and bugs. The crucial role played by the T3D was to provide rapid response, which allowed progressive improvements of the underlying algorithm until the calculation was completed.

### 7.2. *Scalable decomposition for RNA folding*

Dynamic programming, when applied to RNA folding, requires CPU time that scales roughly as the cubic power of the sequence length, and memory that scales quadratically with sequence length. Nonetheless, the basic philosophy driving RNA folding methods on massively parallel platforms is the point of view that memory is the fundamental resource bottleneck, rather than computational speed.

Even though CPU time grows as the cubic power of chain length, sequences such as HIV that are approximately 10 000 nucleotides in length still require only on the order of 35 min to fold on 256 nodes of the Intel `Delta` supercomputer. The same calculation would require of the order of 60 h on a high-end workstation. While this time is hardly ideal, it is certainly an acceptable turn-around time given that there are only a limited number of RNA molecules available to fold. The implementation is thus suitable as a routine tool allowing for a comparative analysis of the complete set of available sequence data for RNA viruses.

On the other hand, memory requirements are a severe problem for RNA molecules the size of HIV. The simplest RNA folding calculation, which computes just the single minimum free energy structure, requires of the order of 1 Gigabyte of memory for a sequence

of the length of HIV-1. More sophisticated algorithms that compute averages over a much larger number of structures near the minimum free energy typically require upwards of 2 Gigabytes. Distributed massively parallel architectures can easily satisfy these memory requirements for viruses such as HIV. Furthermore, future generations of such machines will provide scalable memory enabling even longer sequences to be studied. These scalable resources simply cannot be matched by conventional architectures, and are the primary reason that scalable architectures are necessary for performing RNA folding computations on large macromolecules.

As a consequence of the additivity of the energy contributions, the minimum free energy of an RNA sequence can be calculated recursively by dynamic programming [24]. This method is at the heart of the approach. The basic logic of this scheme is derived from sequence alignment: In fact, folding of RNA can be regarded as a form of alignment of the sequence to itself. The implementation of sequence alignment algorithms on massively parallel architectures is discussed in detail in [13]. The parallel decomposition of the problem is performed by first writing a given sequence along each axis of a two-dimensional matrix. The method can be parallelized because of the fact that all entries along a diagonal $d$ in this array are independent of each other and can therefore be computed concurrently, as in the case of sequence alignment. The major computational difficulty in the case of folding, distinguishing it from standard sequence alignment, is the fact that each entry in diagonal $d$ requires the *explicit* knowledge of a large number of the previously computed matrix entries.

On architectures with distributed memory the basic parallel decomposition consists of assigning the matrix entries within each diagonal evenly. The *efficiency* of the parallelization is measured by

$$\mathcal{E}(N) := \frac{T^*}{Nt},$$

where $T^*$ is the single node execution time, $N$ the number of nodes used for the calculation and $t$ is real time used for the folding (including the backtracking step). The data in Fig. 4 show that efficiencies of more than 90% are obtained even when the smallest possible number of nodes is used for the computation.

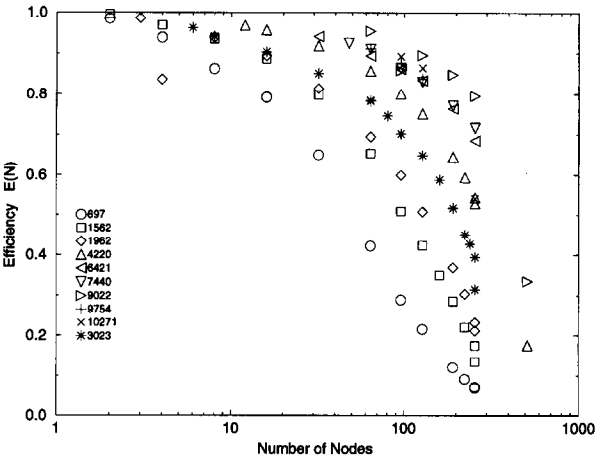



Fig. 4. Scaling efficiency on the Intel Delta for the minimum energy folding of several RNA sequences of varying length (labeled by number of nucleotides).

Since the number of message passing events is $O(nN)$, with the size of the messages not depending on the number of nodes $N$, we expect that to first order the execution time will depend on $N$ as

$$T = T^* + \alpha N + \cdots$$

with a coefficient $\alpha$ which depends in turn on the chain length $n$. Consequently, the efficiency should decrease linearly for moderately small $N$. This is observed in fact. As expected, the efficiency loss due to the parallelization decreases with increasing chain length. It is clear that the method works well on this architecture. The entire approach is very important in the datamining context because comparative analysis of structure from many different sequences is necessary to study highly varying genomes such as HIV. Clustering and alignment methods must be performed on both sequences and predicted structures to identify conserved motifs and other evidence of important structure. A fast folding technique is the crucial bottleneck to allowing comparitive analysis of this type across many sequences, and illustrates clearly the need for high-performance machines in this domain.

## 8. Research challenges

Successful KDD applications continue to appear, driven mainly by a glut in databases that have clearly grown to surpass raw human processing abilities. For examples of success stories in applications in industry see [2] and in science analysis [6]. More detailed case studies are found in [7]. Driving the healthy growth of this field are strong forces (both economic and social) that are a product of the data overload phenomenon. We view the need to deliver workable solutions to pressing problems as a very healthy pressure on the KDD field. Not only will it ensure our healthy growth as a new engineering discipline, but it will provide our efforts with a healthy dose of reality checks; insuring that any theory or model that emerges will find its immediate real-world test environment.

The fundamental problems are still as difficult as they have been for the past few centuries as people considered difficulties of data analysis and how to mechanize it. The challenges facing advances in this field are formidable. Some of them include:

1. Develop mining algorithms for classification, clustering, dependency analysis, and change and deviation detection that scale to large databases. There is a trade-off between performance and accuracy as one surrenders to the fact that data resides primarily on disk or on a server and cannot fit in main memory.
2. Develop schemes for encoding "metadata" (information about the content and meaning of data) over data tables so that mining algorithms can operate meaningfully on a database and so that the KDD system can effectively ask for more information from the user.
3. While operating in a very large sample size environment is a blessing against overfitting problems, data mining systems need to guard against fitting models to data by chance. This problem becomes significant as a program explores a huge search space over many models for a given data set.
4. Develop effective means for data sampling, data reduction, and dimensionality reduction that operate on a mixture of categorical and numeric data fields. While large sample sizes allow us to handle higher dimensions, our understanding of high-dimensional spaces and estimation within them is still fairly primitive. The curse of dimensionality is still with us.
5. Develop schemes capable of mining over inhomogenous data sets (including mixtures of multimedia, video, and text modalities) and deal with sparse relations that are only defined over parts of the data.
6. Develop new mining and search algorithms capable of extracting more complex relationships between fields and able to account for structure over the fields (e.g., hierarchies, sparse relations); i.e., go beyond the flat file or single table assumption.
7. Develop data mining methods that account for prior knowledge of data and exploit such knowledge in reducing search, that can account for costs and benefits, and that are robust against uncertainty and missing data problems. Bayesian methods and decision analysis provide the basic foundational framework.
8. Enhance database management systems to support new primitives for the efficient extraction of necessary-sufficient statistics as well as more efficient sampling schemes. This includes providing SQL support for new primitives that may be needed (e.g. [10]).
9. Scale methods to parallel databases with hundreds of tables, thousands of fields, and terabytes of data. Issues of query optimization in these settings are fundamental.
10. Account for and model comprehensibility of extracted models; allow proper trade-offs between complexity and understandability of models for purposes of visualization and reporting; enable interactive exploration where the analyst can easily provide hints to help the mining algorithm with its search.
11. Develop theory and techniques to model growth and change in data. Large databases, because they grow over a long time, do not typically grow as if sampled from a static joint probability density. The question of how does the data grow? needs to be better understood (see articles by P. Huber,

by Fayyad and Smyth, and by others in [15]) and tools for coping with it need to be developed.

12. Incorporate basic data mining methods with "any-time" analysis that can quantify and track the trade-off between accuracy and available resources, and optimize algorithms accordingly.
13. Test scalable statistics methods in HPC environments. We will almost certainly find the unexpected when current methods, useful on small to moderate size data, are implemented on massive databases. New techniques based on these insights will need to be constructed to deal with the regime of large numbers of data points and large dimensionality.
14. Construct balanced systems emphasizing the importance of scalable I/O and memory bandwidth as well as traditional CPU and RAM drivers. In some cases, out-of-core solutions to important problems may be inevitable, and will require substantial effort.

## 9. Concluding remarks

KDD holds the promise of an enabling technology that could unlock the knowledge lying dormant in huge databases. Perhaps the most exciting aspect is the possibility of the birth of a new research area properly mixing statistics, databases, automated data analysis and reduction, and other related areas. The mix could produce a new breed of algorithms, tuned to working on large databases and scalable both in data set size and in parallelism.

In this paper, we provided an overview of this area, defined the basic terms, and covered some sample case studies of applications in science data analysis. We focussed on the role of high-performance computing and on notions related to parallelizing some data mining algorithms, and concluded by listing future challenges for data mining and KDD. While KDD will draw on the substantial body of knowledge built up in its constituent fields, it is our hope that a new science will inevitably emerge as these challenges are addressed, and that suitable mixtures of ideas from each discipline will transform our ability to exploit massive and ever-changing data sets.

## References

[1] J. Aubele, L. Crumpler, U. Fayyad, P. Smyth, M. Burl and P. Perona, Locating small volcanoes on venus using a scientist-trainable analysis system, in: *Proc. 26th Lunar and Planetary Science Conf.*, 1458 (LPI/USRA, Houston, TX, 1995).

[2] R. Brachman, T. Khabaza, W. Kloesgen, G. Piatetsky-Shapiro and E. Simoudis, Industrial applications of data mining and knowledge discovery, *Commun. ACM* 39 (11) (1996).

[3] M.C. Burl, U. Fayyad, P. Perona, P. Smyth and M.P. Burl, Automating the hunt for volcanoes on venus, in: *Proc. Computer Vision and Pattern Recognition Conf. (CVPR-94)* (IEEE Computer Soc. Press, Silver Spring, MD, 1994) 302–308.

[4] E.F. Codd, Providing OLAP (On-line analytical processing) to User-Analysts: An IT Mandate, E.F. Codd and Associates (1993).

[5] R.O. Duda and P.E. Hart, *Pattern Classification and Scene Analysis* (Wiley, New York, 1973).

[6] U. Fayyad, D. Haussler and P. Stolorz, Mining science data, *Commun. of ACM* 39 (11) (1996).

[7] U. Fayyad, G. Piatetsky-Shapiro, P. Smyth and R. Uthurusamy, eds., *Advances in Knowledge Discovery and Data Mining* (MIT Press, Cambridge, MA, 1996).

[8] C. Glymour, D. Madigan, D. Pregibon and P. Smyth, Statistical themes and lessons for data mining, *Data Mining and Knowledge Discovery* 1 (1) (1997).

[9] C. Glymour, R. Scheines, P. Spirtes and K. Kelly, *Discovering Causal Structure* (Academic Press, New York, 1987).

[10] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow and H. Pirahesh, Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub totals, *Data Mining and Knowledge Discovery* 1 (1) (1997).

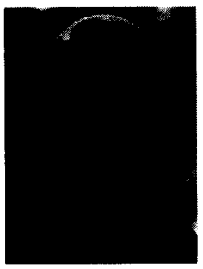
[11] D. Heckerman, Bayesian networks for data mining, *Data Mining and Knowledge Discovery* 1 (1) (1997).

[12] I. Hofacker, M. Huynen, P. Stadler and P. Stolorz, Knowledge discovery in RNA sequence families of HIV using scalable computers, *Proc. 2nd Internat. Conf. on Knowledge Discovery and Datamining* (AAAI Press, Menlo Park, CA, 1996) 20–25.

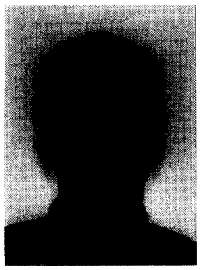
[13] R. Jones, Protein sequence and structure aligments on massively parallel computers, *Internat. J. Supercomp. Appl.* 6 (1992) 138–146.

[14] J.D. Kennefick, R.R. De Carvalho, S.G. Djorgovski, M.M. Wilber, E.S. Dickinson, N. Weir, U. Fayyad and J. Roden, The discovery of five quasars at $z > 4$ using the second palomar sky survey, *Astronomical J.* 110 (1) (1995) 78–86.

[15] J. Kettenring and D. Pregibon, eds., *Statistics and Massive Data Sets, Report to the Committee on Applied and Theoretical Statistics* (National Research Council, Washington, DC, 1996).
[16] E.E. Leamer, *Specification Searches: Ad hoc Inference with Nonexperimental Data* (Wiley, New York, 1978).
[17] M. Mehta, R. Agrawal and J. Rissanen, SLIQ: A fast scalable classifier for data mining, *Proc. EDBT-96* (Springer, Berlin, 1996).
[18] D. Pfitzner and J. Salmon, Parallel halo finding in $N$-body cosmology simulations, *Proc. 2nd Internat. Conf. on Knowledge Discovery and Datamining* (AAAI Press, Menlo Park, CA, 1996) 208–213.
[19] G. Piatetsky-Shapiro and W. Frawley, eds., *Knowledge Discovery in Databases* (MIT Press, Cambridge, MA, 1991).
[20] A. Silberschatz and A. Tuzhilin, On subjective measures of interestingness in knowledge discovery, in: *Proc. KDD-95: 1st Internat. Conf. on Knowledge Discovery and Data Mining* (AAAI Press, Menlo Park, CA, 1995) 275–281.
[21] P. Stolorg and C. Dean, Quakefinder: A scalable data mining system for detecting earthquakes from space, in: *Proc. 2nd Internat. Conf. on Knowledge Discovery and Data Mining* (AAAI Press, Menlo Park, CA, 1996) 208–213.
[22] P. Stolorz et al., Fast spatio-temporal data mining from large geophysical datasets, *Proc. 1st Internat. Conf. on Knowledge Discovery and Datamining* (AAAI Press, Menlo Park, CA, 1995) 300–305.
[23] J.W. Tukey, *Exploratory Data Analysis* (Addison-Wesley, Reading, MA, 1975).
[24] M.S. Waterman, Secondary structure of single-stranded nucleic acids, *Studies on Foundations and Combinatorics, Advances in Mathematics Supplementary Studies* (Academic Press, New York, 1978).
[25] N. Weir, U. Fayyad and S.G. Djorgovski, Automated star/galaxy classification for digitized POSS-II, *Astronomical J.* 109 (6) (1995) 2401–2412.
[26] N. Weir, S.G. Djorgovski and U.M. Fayyad, Initial galaxy counts from digitized POSS-II, *Astronomical J.* 110 (1) (1995) 1–20.



**Usama Fayyad** is a Senior Researcher at Microsoft Research (see the URL http://www.research.microsoft.com/~fayyad). His interests include knowledge discovery in large databases, data mining, machine learning, statistical pattern recognition, and clustering. After receiving the Ph.D. degree in 1991, he joined the Jet Propulsion Laboratory (JPL), California Institute of Technology (until 1996). At JPL, he headed the Machine Learning Systems Group where he developed data mining systems for analysis of large scientific databases. He remains affiliated with JPL as a distinguished visiting scientist. Fayyad received the JPL 1993 Lew Allen Award for Excellence in Research, and the 1994 NASA Exceptional Achievement Medal. He was program co-chair of KDD-94 and KDD-95 (*the 1st Internat. Conf. on Knowledge Discovery and Data Mining*). He is general chair of KDD-96, an editor-in-chief of the journal: *Data Mining and Knowledge Discovery* (http://www.research.microsoft.com/datamine), and co-editor of the book: *Advances in Knowledge Discovery and Data Mining* (MIT Press, Cambridge, MA, 1996).



**Paul E. Stolorz** is Technical Group Supervisor of the Machine Learning Systems Group at JPL (http://www-aig.jpl.nasa.gov/mls). His research interests include scalable datamining, machine learning, statistical pattern recognition, computational biology and biochemistry, and plate tectonics. He holds a Ph.D. in theoretical physics from the California Institute of Technology, and a DIC from Imperial College, University of London. Stolorz has been involved in scientific applications of machine learning for over 12 years, and in massively parallel computing for over 15 years, with numerous archival publications in the scientific and machine learning literature. He regularly lectures and conducts tutorials on the applications of machine learning to large-scale scientific problems. He serves on the editorial board of the journal: *Data Mining and Knowledge Discovery*. He has served on the Program Committee for KDD-96 and KDD-97, *the 2nd Internat. Conf. on Knowledge Discovery and Data Mining*, and is program co-chair for KDD-98.